

Understanding JDK, SDK, the Java Compiler, and Maven Language Versions

When you build a Spring Boot project, several tools work together before your application ever starts. Names like **JDK**, **SDK**, **compiler**, and **Maven** appear in project settings and build files, and it is easy to mix them up. This article explains what each one is and how they relate.

1. What is the JDK?

The **JDK** (Java Development Kit) is the full toolkit for writing and running Java applications. It typically includes:

- `javac` — the Java compiler
- **JVM** — the runtime that executes compiled programs
- **Standard libraries** — the built-in Java APIs
- Supporting tools for packaging and debugging

When people say “I installed Java 22,” they usually mean they installed a **JDK 22**. That installation is a concrete set of programs and libraries on disk. Different machines (or even the same machine) can have several JDKs installed side by side: JDK 11, JDK 17, JDK 22, and so on.

The JDK version matters because newer JDKs support newer language features.

2. What is the SDK (in IntelliJ)?

SDK stands for Software Development Kit. In general software, an SDK is a broader package of tools for building software.

In **IntelliJ IDEA**, the term **Project SDK** almost always means: *which JDK should this project use?*

So in practice:

Term	Meaning in a Java IDE project
JDK	The actual Java installation on your machine
Project SDK	The IDE’s choice of which JDK to use for this project

If IntelliJ says **Project SDK = openjdk-22**, it is telling the IDE: use JDK 22 to compile and run this project. The SDK is not a second compiler. It is a pointer to a JDK.

3. Who is the compiler?

The compiler is `javac`, a program that lives inside the JDK.

Its job is simple to describe and essential in practice:

```
graph TD
    A[Your source code (.java)] --> B[javac (compiler)]
    B --> C[Bytecode (.class)]
    C --> D[JVM runs the application]
```

You write human-readable Java source files. The compiler turns them into `.class` files that the JVM can execute. IntelliJ does not invent its own Java language compiler for ordinary builds; it invokes `javac` from the selected Project SDK (JDK).

Maven builds do the same thing: they also call a Java compiler, usually from whatever JDK Maven is running with.

4. Language version: why “21” or “22” matter

Java has evolved through many versions: 8, 11, 17, 21, 22, and beyond. When you compile, you are not only choosing *which JDK runs the compiler*; you also choose *which language level the source is treated as*.

Common settings in Maven are:

- `java.version`
- `maven.compiler.source`
- `maven.compiler.target`
- `maven.compiler.release`

These tell the build: “Treat this project as Java 22” (or 21, or another version).

A healthy project keeps the Project SDK and the Maven language version aligned. For example, if the IDE uses JDK 22, the build should also declare Java 22.

5. Where does Maven fit in?

Maven is the build tool. Its configuration file is `pom.xml` .

Maven decides:

- which libraries your project depends on
- how the project is compiled and packaged
- which Java language version the compiler should use

Spring Initializr often sets something like:

```
<java.version>21</java.version>
```

That property is intended to tell Spring Boot and the Maven compiler plugin which Java version to use. When everything is wired correctly, IntelliJ’s Project SDK and Maven’s language version should agree.

6. How the pieces fit together

Here is the full picture in one place:

Piece	Role
JDK	The installed Java toolkit (includes <code>javac</code> and the JVM)
Project SDK	IntelliJ’s selected JDK for the project
Compiler (<code>javac</code>)	Turns <code>.java</code> into <code>.class</code>
Maven + <code>pom.xml</code>	Build tool and configuration, including language version
Language version (21/22)	Which Java level the source should be compiled as

A healthy setup looks like this:

1. Project SDK points to a modern JDK (for example, 22).
2. `pom.xml` clearly says the project uses the same modern language level.

When those points are aligned, the project compiles and runs smoothly.

Conclusion

JDK is the real Java toolkit. **SDK**, in IntelliJ, is usually just the project's chosen JDK. The **compiler** is `javac`, and it is the program that turns source code into bytecode. **Maven** tells that compiler which language version to use.

Once you see these roles clearly, Java tooling becomes much less mysterious: the tools are not competing systems, but layers that must agree on one simple question — *which Java version are we building with?*